

The F2Z polynomial commitment scheme

Relations, reductions, and a normative prover–verifier specification

F2Z is a polynomial-commitment layer for proving the Lin-BitsDual relation. Given a public vector $\mathbf{v}$ and a value $\mu \in \mathbb{F}_q$, the relation asserts

$$\langle \mathbf{v}, \pi_q(\mathbf{h}) \rangle_{\mathbb{F}_q} = \mu, \quad \pi_2(\mathbf{h}) = \mathbf{M}\pi_2(\mathbf{f}), \quad O = \text{Enc}_C(\text{Pack}(\pi_2(\mathbf{f}))),$$

where $\mathbf{f}$ is the committed binary witness, $\mathbf{h}$ is the virtual witness used by the PIOP, and $\mathbf{M}$ is the effective virtualization map. Virtualization is optional: when the developer omits it, $\mathbf{h} = \mathbf{f}$ and $\mathbf{M} = I$. Here $\pi_2 : \mathbb{Z} \rightarrow \mathbb{F}_2$ denotes coordinatewise reduction modulo 2. The PIOP supplies this linear claim to F2Z.

Drawing inspiration from binary-field techniques used in systems such as Flock and Binius64, F2Z combines the following techniques:

- **Binary packing.** The committed witness bits are packed into elements of the binary extension field $\mathbb{K}$ and encoded using the binary linear code $\mathcal{C}$. This allows the commitment and opening protocols to operate on packed field elements.
- **Witness virtualization.** When configured, the public map $\mathbf{M}$ relates the committed witness $\mathbf{f}$ to the PIOP-facing witness $\mathbf{h}$. This permits a constraint-specific witness layout while retaining one commitment; the direct configuration uses the identity map.
- **Input-claim shaping.** A degree-two LinCheck over $\mathbb{F}_q$ reduces the arbitrary public $\mathbf{v}$ -weighted claim to one evaluation-shaped claim. The resulting row and column equality weights are verifier-derived and are the only factors consumed by the integer-fold core.
- **Bounded integer lifting.** Coefficients in $\mathbb{F}_q$ are lifted to their canonical integers. The configured row dimension guarantees that every resulting integer fold lies within the injective range of the binary-field exponent encoding.
- **Grand-product encoding.** Because the witness coordinates are binary, each bounded linear fold can be expressed as a product identity over $\mathbb{K}$. This converts the original linear claim into product claims suited to binary-field verification.
- **GKR and LinCheck reductions.** GKR verifies the product computation succinctly, while the row LinCheck reduces the resulting weighted claim to a multilinear-extension evaluation of the virtual witness.
- **Adjoint pullback.** The verifier transports the final coefficient through $\mathbf{M}^\top$, converting a claim on $\mathbf{h}$ into a claim on the originally committed witness $\mathbf{f}$ without another commitment or protocol message. This step is the identity when virtualization is omitted.
- **Structured ring switching.** The pulled-back coefficient is decomposed across the 128-bit packing boundary. The ring switch then converts the resulting inner product on $\mathbf{f}$ directly into one packed linear claim for the recursive Ligerito opening; no post-adjoint LinCheck is required.

Together, these techniques allow F2Z to prove inner-product claims over $\mathbb{F}_q$, and therefore support constraints beyond $\mathbb{F}_2$, while retaining a binary-field commitment. This specification fixes the Lin-BitsDual relation, its prover–verifier reductions, and its concrete transcript and encoding rules.

1 Overview and interfaces

1.1 Committed and virtual witness views

The PCS receives a committed binary witness $\mathbf{f}$. Before proving, the developer fixes the matrix layouts used by the commitment and the PIOP. The same configuration may optionally include a public virtualization map:

- **Direct.** The virtualization map is omitted and the PIOP uses the committed witness directly, so $\mathbf{h} = \mathbf{f}$.
- **Virtualized.** The PIOP uses a binary witness $\mathbf{h}$ related to the committed witness by a nontrivial developer-selected public matrix $\mathbf{M}$.

Throughout, $\mathbf{f}$ is the commitment-facing vector and $\mathbf{h}$ is the PIOP-facing vector. When $\mathbf{M}$ is omitted, every uniform formula below interprets it as the identity map.

The virtual witness is

$$\pi_2(\mathbf{h}) = \mathbf{M}\pi_2(\mathbf{f}). \quad (1)$$

Here $\pi_2 : \mathbb{Z} \rightarrow \mathbb{F}_2$, $x \mapsto x \bmod 2$, acts coordinatewise. Since $\mathbf{f}$ and $\mathbf{h}$ are binary, Equation (1) uniquely determines $\mathbf{h}$. If virtualization is enabled, the developer fixes $\mathbf{M}$ before proving; it is not chosen by the prover.

Only $\mathbf{f}$ is committed. The developer-configured matrices F and H are reshaping of $\mathbf{f}$ and $\mathbf{h}$, not additional witnesses. The commitment is

$$\mathbf{f} \in \{0, 1\}^{n_f}, \quad \text{vec}(F) := \pi_2(\mathbf{f}), \quad P := \text{Pack}(F), \quad O := \text{Enc}_C(P), \quad \rho := \text{Root}(O). \quad (2)$$

Neither H , the layout of $\mathbf{h}$, nor any value derived from H is encoded in O .

Adjoint convention. For the standard inner product over a field $\mathbb{E}$, the adjoint of a matrix $\mathbf{T}$ is its transpose $\mathbf{T}^\top$:

$$\langle \mathbf{u}, \mathbf{T}\mathbf{v} \rangle_{\mathbb{E}} = \langle \mathbf{T}^\top \mathbf{u}, \mathbf{v} \rangle_{\mathbb{E}}. \quad (3)$$

Throughout this specification, applying the adjoint means using this identity to move a public matrix across the inner product. It is a deterministic rewrite and sends no protocol message.

1.2 PIOP linear-claim handoff

The end-to-end proof begins with a constraint-satisfaction relation over $\mathbb{F}_q$ evaluated on the virtual witness $\pi_q(\mathbf{h})$. The PIOP reduces that relation to a general Lin-BitsDual claim consumed by F2Z. The input Sumcheck phase of Π_1 shapes that claim before the integer-fold phase:

$$\begin{aligned} (x; \pi_q(\mathbf{h})) \in \mathcal{R}_{CS} &\xrightarrow{\Pi_{\text{PIOP}}} \langle \mathbf{v}, \pi_q(\mathbf{h}) \rangle_{\mathbb{F}_q} = \mu \\ &\xrightarrow{\Pi_1} \mathcal{R}_{\text{PESAT}}[\mathbf{h}] \\ &\xrightarrow{\text{F2Z}} \text{opening against } \rho. \end{aligned} \quad (4)$$

The PIOP is defined by the constraint system and is not part of F2Z itself. The end-to-end interaction includes it as the subprotocol that produces the F2Z handoff. F2Z receives the arbitrary public coefficient $\mathbf{v}$ and does not inspect the equations or reductions that produced it. The checked input Sumcheck is internal to Π_1 ; it does not factor $\mathbf{v}$, but replaces the original scalar claim by an evaluation-shaped intermediate claim before producing F2Z PESAT instances.

2 Instance and parameters

Section 2 is the canonical registry for persistent and cross-phase F2Z notation. A definition gives the exact value denoted by a symbol, its type gives the ambient set or interface class, and its rule records the normative constraint on its use. Section 3 specifies how the transcript derives the non-input symbols registered here.

2.1 Algebraic domains and canonical maps

Symbol	Definition	Type	Rule
$\mathbb{Z}$	Exact integer domain	Ring [John: don't need]	
$\mathbb{F}_2$	Binary field	$\mathbb{F}_2$	
$\mathbb{F}_q$	$q = 2^{100} - 15$	Finite field	
$\mathbb{K}$	$\mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1)$	$\mathbb{F}_{2^{128}}$	Used for packing, products, GKR, and openings.
π_2	$x \mapsto x \bmod 2$	$\mathbb{Z} \rightarrow \mathbb{F}_2$	
$\text{lift}_2(b)$	The unique $u \in \{0, 1\} \subset \mathbb{Z}$ satisfying $\pi_2(u) = b$	$\mathbb{F}_2 \rightarrow \{0, 1\}$	Canonical bit representative; not an inverse of π_2 on all of $\mathbb{Z}$.
π_q	$x \mapsto x \bmod q$	$\mathbb{Z} \rightarrow \mathbb{F}_q$	
lift_q	$\text{lift}_q(a)$ is the unique integer in $[0, q)$ satisfying $\pi_q(\text{lift}_q(a)) = a$	$\mathbb{F}_q \rightarrow [0, q)$	Canonical integer lift; no alternative representative is accepted.
$\iota_{2 \rightarrow \mathbb{K}}$	Canonical subfield embedding	$\mathbb{F}_2 \hookrightarrow \mathbb{K}$	Applied coordinatewise when a binary witness enters a $\mathbb{K}$ -claim.
β_v	X^v in the fixed polynomial basis of $\mathbb{K}$	Element of $\mathbb{K}$	$0 \leq v < 128$, ordered by increasing v .

2.2 Dimensions and index conventions

Symbol	Definition	Type	Rule
n_f	Length of the committed witness	$\mathbb{N}$	$n_f = 2^m f$ and $128 \mid n_f$.
n_h	Length of the virtual witness	$\mathbb{N}$	$n_h = 2^m h$.
m_f	Boolean dimension of $\mathbf{f}$	$\mathbb{N}$	
m_h	Boolean dimension of $\mathbf{h}$	$\mathbb{N}$	
$d_{F,r}, d_{F,c}$	Row and column dimensions of F	$\mathbb{N}^2$	$d_{F,r} + d_{F,c} = m_f$.
d_r, d_c	Row and column dimensions of H	$\mathbb{N}^2$	$d_r + d_c = m_h$ and $2^{d_r} q < 2^{127}$.
m_p	Packed-message dimension	$\mathbb{N}$	$m_p = m_f - 7$.
r	Row index of H	$0 \leq r < 2^{d_r}$	
c	Column index of H	$0 \leq c < 2^{d_c}$	
j	Packed-element index	$0 \leq j < n_f/128$	
v	Basis-coordinate index in K	$0 \leq v < 128$	index to reference a coefficient for a packed galios number

2.3 Witnesses, layouts, and virtualization

Symbol	Definition	Type	Rule
$\mathbf{f}$	Commitment-facing binary witness	$\{0, 1\}^{n_f}$	The PCS commits only to this vector.
F	Developer-configured reshaping of $\pi_2(\mathbf{f})$	$\mathbb{F}_2^{2^{d_{F,r}} \times 2^{d_{F,c}}}$	$\text{vec}(F) = \pi_2(\mathbf{f})$. The developer reshapes it in terms of $2^{d_{F,r}} \times 2^{d_{F,c}}$.
$\mathbf{h}$	PIOP-facing binary witness	$\{0, 1\}^{n_h}$	Equals $\mathbf{f}$ when virtualization is omitted.
H	Developer-configured reshaping of $\pi_2(\mathbf{h})$	$\mathbb{F}_2^{2^{d_r} \times 2^{d_c}}$	$\text{vec}(H) = \pi_2(\mathbf{h})$.
$H_{\mathbb{Z}}$	Entrywise $\text{lift}_2(H)$	$\{0, 1\}^{2^{d_r} \times 2^{d_c}} \subset \mathbb{Z}^{2^{d_r} \times 2^{d_c}}$	Used in exact integer folds.
H_q	Entrywise $\pi_q(H_{\mathbb{Z}})$	$\mathbb{F}_q^{2^{d_r} \times 2^{d_c}}$	Used in the Lin-BitsDual claim and the input Sumcheck.
$H_{\mathbb{K}}$	Entrywise $\iota_{2 \rightarrow \mathbb{K}}(H)$	$\mathbb{K}^{2^{d_r} \times 2^{d_c}}$	Used in PESAT, GKR, and MLE claims.
vec	Flattening of a configured matrix layout	Matrix-to-vector map	The developer fixes its coordinate order before proving.
$\mathbf{M}$	Effective virtualization map	$\mathbb{F}_2^{n_h \times n_f}$	Supplied by the developer when virtualization is enabled. Otherwise $n_h = n_f$, both layouts preserve the same flattened coordinate order, $\mathbf{h} = \mathbf{f}$, and $\mathbf{M} := I$.
$\mathbf{M}_{\mathbb{K}}$	Scalar extension of the effective $\mathbf{M}$	$\mathbb{K}^{n_h \times n_f}$	Its transpose is the adjoint from Equation (3).
$\mathbf{f}_{\mathbb{K}}, \mathbf{h}_{\mathbb{K}}$	Coordinatewise images under $\iota_{2 \rightarrow \mathbb{K}}$	$\mathbb{K}^{n_f}$ and $\mathbb{K}^{n_h}$	Used only in $\mathbb{K}$ -linear claims.

The matrices F and H are views of the witness vectors, not additional witnesses. Their shapes and coordinate orders are developer configuration choices fixed before proving. The configured Boolean address of H is $z = (r, c) \in \{0, 1\}^{d_r} \times \{0, 1\}^{d_c}$; its vectorization and every multilinear extension in this specification use this same row/column bit split. In particular, reshaping a vector $\mathbf{u} \in \mathbb{F}_q^{n_h}$ into a matrix U means $\text{vec}(U) = \mathbf{u}$

under that configured order, not an independently chosen row-major convention.

2.4 Packing and commitment

Symbol	Definition	Type	Rule
Pack	Packs consecutive groups of 128 coordinates into $\mathbb{K}$	$\mathbb{F}_2^{nf} \rightarrow \mathbb{K}^{nf/128}$	Uses the ordered basis $(\beta_v)_{v < 128}$.
P	$P[j] = \sum_{v=0}^{127} \text{vec}(F)[128j + v]\beta_v$	$\mathbb{K}^{nf/128}$	Derived only from the configured view F .
F_v	$F_v[j] = \text{vec}(F)[128j + v]$	$\mathbb{F}_2^{mp}$	The v -th bit slice of the packed message, so $P[j] = \sum_v F_v[j]\beta_v$.
$\mathcal{C}$	Configured encoding code	Public code	
$\text{Enc}_{\mathcal{C}}$	Canonical encoder for $\mathcal{C}$	$\mathbb{K}^{nf/128} \rightarrow \mathbb{K}^{nO}$	
O	$\text{Enc}_{\mathcal{C}}(P)$	$\mathbb{K}^{nO}$	Encodes P and never the virtual view H .
Root	Configured Merkle-root algorithm	$\mathbb{K}^{nO} \rightarrow \{0, 1\}^\lambda$	Constructs a Merkle tree on $\text{Enc}_{\mathcal{C}}(P)$.
ρ	$\text{Root}(O)$	$\{0, 1\}^\lambda$	Public commitment to O .

2.5 Structured ring-switch coefficients

Symbol	Definition	Type	Rule
$A_f[v, j]$	$\mathbf{a}_f[128j + v]$ in the configured commitment order	$\mathbb{K}^{128 \times 2^{mp}}$	Pack-boundary view of the public coefficient produced by the adjoint.
Θ	$\{0, \dots, t_{\text{sw}} - 1\}$ in canonical order	Finite nonempty index set	Derived by the developer configuration from $\mathbf{a}_f$, with $t_{\text{sw}} < 2^{32}$.
A_θ, B_θ	$A_\theta[v] \in \mathbb{K}$ and $B_\theta[j] \in \mathbb{K}$	$\mathbb{K}^{128}$ and $\mathbb{K}^{2^{mp}}$	Must satisfy $A_f[v, j] = \sum_{\theta \in \Theta} A_\theta[v]B_\theta[j]$ for every (v, j) .
RingDecomp	$\mathbf{a}_f \mapsto (\Theta, (A_\theta, B_\theta)_{\theta \in \Theta})$	Public deterministic map	The verifier derives the same ordered decomposition; no proof field is used.

Every coefficient admits the canonical fallback $\Theta = \{0, \dots, 127\}$, $A_\theta[v] = 1$ when $v = \theta$ and 0 otherwise, and $B_\theta[j] = A_f[\theta, j]$. A developer configuration may instead provide a smaller structured decomposition, such as the translated-equality terms induced by rotations, shifts, or other public witness wiring.

2.6 Lin-BitsDual public claim

Symbol	Definition	Type	Rule
$\mathbf{v}$	Public coefficient vector	$\mathbb{F}_q^{nh}$	Uses the configured coordinate order of H .
V	$\text{vec}(V) = \mathbf{v}$ in the configured order of H	$\mathbb{F}_q^{2^{dr} \times 2^{dc}}$	Arbitrary; no rank or separability condition is imposed.
μ	Claimed inner product $\langle \mathbf{v}, \pi_q(\mathbf{h}) \rangle_{\mathbb{F}_q} = \langle \mathbf{v}, \text{vec}(H_q) \rangle_{\mathbb{F}_q}$	$\mathbb{F}_q$	Initial target of the input Sumcheck.
$\mathcal{I}_{\text{LinBitsDual}}$	$(\mathbf{v}, \mu)$ together with the developer configuration	Public F2Z instance	Binds the arbitrary input claim, both layouts, and the effective virtualization map.
$\mathbf{s}_h = (\mathbf{s}_r, \mathbf{s}_c)$	Input-Sumcheck terminal point	$\mathbb{F}_q^{mh}$	Transcript-derived with $ \mathbf{s}_r = d_r$ and $ \mathbf{s}_c = d_c$.
κ	$\widetilde{\mathbf{v}}(\mathbf{s}_h)$	$\mathbb{F}_q$	Computed by the verifier; never supplied by the prover.
τ	Input-Sumcheck terminal value	$\mathbb{F}_q$	Reduced target satisfying $\kappa \widetilde{H}_q(\mathbf{s}_h) = \tau$.

After the input Sumcheck, the verifier locally derives $a_r := \text{eq}_{\mathbb{F}_q}(r, \mathbf{s}_r)$ and $a'_c := \kappa \text{eq}_{\mathbb{F}_q}(c, \mathbf{s}_c)$. Consequently,

$$W[r, c] := \kappa \text{eq}_{\mathbb{F}_q}((r, c), \mathbf{s}_h) = \text{eq}_{\mathbb{F}_q}(r, \mathbf{s}_r) \kappa \text{eq}_{\mathbb{F}_q}(c, \mathbf{s}_c) = a_r a'_c, \quad (5)$$

only the new coefficient W is separable. Equation (5) is a consequence of the equality polynomial at the Sumcheck terminal point; it is not a decomposition of the arbitrary input matrix V .

2.7 Integer lifts and fold parameters

Symbol	Definition	Type	Rule
γ_r	$\text{lift}_q(a_r)$	$[0, q) \subset \mathbb{Z}$	Unique canonical integer lift of a_r .
$\widehat{\mu}_c$	$\sum_{r < 2^{dr}} \gamma_r H_{\mathbb{Z}}[r, c]$	$[0, 2^{127}) \subset \mathbb{Z}$	The bound follows from $2^{dr}q < 2^{127}$ and is required for exponent injectivity.
μ_c	Prover-supplied claimed fold	$[0, 2^{127}) \subset \mathbb{Z}$	Accepted only if its exponent matches PESAT and $\tau = \sum_c a'_c \pi_q(\mu_c)$.
g	Product generator	$\mathbb{K}^\times$	$\text{ord}(g) = 2^{128} - 1$, making exponentiation injective on accepted folds.
ν_c	g^{μ_c}	$\mathbb{K}^\times$	Must equal $\prod_r (1 + (g^{\gamma_r} - 1)H_{\mathbb{K}}[r, c])$.

2.8 Protocol interfaces

Symbol	Definition	Rule
Π_1	$\begin{aligned} & \langle \mathbf{v}, \pi_q(\mathbf{h}) \rangle_{\mathbb{F}_q} = \mu \\ \implies & g^{\mu c} = \prod_r \left(1 + (g^{\gamma r} - 1) H_{\mathbb{K}}[r, c] \right) \end{aligned}$	Runs the input Sumcheck, derives the canonical integer weights, and checks the fold bounds, exponentiation, and reconstruction of the reduced scalar claim.
Π_{GKR}	$\begin{aligned} g^{\mu c} &= \prod_r \left(1 + (g^{\gamma r} - 1) H_{\mathbb{K}}[r, c] \right) \\ \implies & \sum_z \omega_{\xi, z} H_{\mathbb{K}}[z] = \sigma \end{aligned}$	
Π_{MLE}	$\begin{aligned} & \sum_z \omega_{\xi, z} H_{\mathbb{K}}[z] = \sigma \\ \implies & \tilde{H}_{\mathbb{K}}(\mathbf{r}_h) = \eta \end{aligned}$	Typically reduced by LinCheck_H .
Adj_M	$\langle \mathbf{u}, M_{\mathbb{K}} \mathbf{v} \rangle_{\mathbb{K}} \mapsto \langle M_{\mathbb{K}}^T \mathbf{u}, \mathbf{v} \rangle_{\mathbb{K}}$	
Π_{iopp}	$\begin{aligned} & \langle \mathbf{a}_f, \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}} = \eta \\ \implies & \langle B, \tilde{P} \rangle_{\mathbb{K}} = \beta \implies \text{accept} \end{aligned}$	Performs one ring switch and one recursive Ligerito opening.

3 Relations and direct reductions

3.1 Relations and reduction notation

This section defines the following five relation types, in protocol order:

$$\mathcal{R}_{\text{LinBitsDual}}, \mathcal{R}_{\text{PESAT}}, \mathcal{R}_{\text{sum}}^h, \mathcal{R}_{\text{eval}}^h, \mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}).$$

These relations share the committed witness, encoded oracle, developer configuration, and public root fixed in Sections 1–2.

For reductions, the notation

$$\mathcal{R}_0 \equiv_G \mathcal{R}_1 \quad \text{and} \quad \mathcal{R}_0 \xRightarrow{\Pi} \mathcal{R}_1 \quad (6)$$

distinguishes a public, zero-message rewrite G from an interactive reduction Π . Each subsection defines one relation and then gives the reduction to its immediate successor. The chain stops at the encoded $\mathcal{R}_{\text{Lin}}$ relation; Section 4 opens the resulting claim.

3.2 The Lin-BitsDual relation

F2Z receives an arbitrary linear claim over $\mathbb{F}_q$. Use the canonical $\mathbb{F}_q$ -view $H_q = \pi_q(H_{\mathbb{Z}})$ from Section 2. The input relation is

$$\begin{aligned} \mathcal{R}_{\text{LinBitsDual}}[\mathbf{h}] &:= \left\{ ((\mathbf{v}, \mu, \mathbf{M}), O; \mathbf{f}) : \mathbf{f} \in \{0, 1\}^{nf}, \quad \text{vec}(H) = \mathbf{M} \pi_2(\mathbf{f}), \right. \\ &\quad \mu = \sum_{r \in \{0, 1\}^{d_r}} \sum_{c \in \{0, 1\}^{d_c}} v[(r, c)] H_q[r, c], \\ &\quad \left. \text{Equations (1) and (2) hold} \right\}. \end{aligned} \quad (7)$$

The PIOP that produced this claim is separate from F2Z. From this point onward, the protocol uses only the public Lin-BitsDual instance, the developer configuration, and the commitment to $\mathbf{f}$.

Direct reduction to F2Z PESAT (Π_1). The successor of Lin-BitsDual is PESAT. To retain that boundary while accepting an arbitrary public coefficient, this specification writes $\Pi_1 := \Pi_{1, \text{in}}; \Pi_{1, \text{core}}$. The first phase is the input Sumcheck adapter, and the second is the bounded integer-lifting and grand-product phase corresponding to the paper’s core reduction.

Input Sumcheck phase. Let $\tilde{\mathbf{v}}$ and $\tilde{H}_q$ be the multilinear extensions of the two tables in Equation (7). The incoming scalar claim is the hypercube sum

$$\mu = \sum_{r \in \{0, 1\}^{d_r}} \sum_{c \in \{0, 1\}^{d_c}} \tilde{\mathbf{v}}(r, c) \tilde{H}_q(r, c). \quad (8)$$

A standard m_h -round sumcheck for the individual-degree-two summand $\tilde{\mathbf{v}} \tilde{H}_q$ reduces this claim to

$$\tau = \kappa \tilde{H}_q(\mathbf{s}_h), \quad \kappa := \tilde{\mathbf{v}}(\mathbf{s}_h), \quad \mathbf{s}_h \in \mathbb{F}_q^{m_h}. \quad (9)$$

Here $(\mathbf{s}_h, \tau)$ is transcript-derived and κ is computed by the verifier. No division is required, including when $\kappa = 0$; the remainder of Π_1 reduces this terminal claim to the successor PESAT relation.

For $\mathbb{E} \in \{\mathbb{F}_q, \mathbb{K}\}$, Boolean address $z \in \{0, 1\}^m$, and $\mathbf{x} \in \mathbb{E}^m$, define

$$\text{eq}_{\mathbb{E}}(z, \mathbf{x}) := \prod_{j=0}^{m-1} \left((1 - z_j)(1 - x_j) + z_j x_j \right). \quad (10)$$

We omit the subscript when the field is clear. Writing $\mathbf{s}_h = (\mathbf{s}_r, \mathbf{s}_c)$, set $a_r := \text{eq}_{\mathbb{F}_q}(r, \mathbf{s}_r)$ and $a'_c := \kappa \text{eq}_{\mathbb{F}_q}(c, \mathbf{s}_c)$. Then

$$W[r, c] := \kappa \text{eq}_{\mathbb{F}_q}((r, c), \mathbf{s}_h) = a_r a'_c, \quad \tau = \sum_c a'_c \sum_r a_r H_q[r, c]. \quad (11)$$

Thus only the derived equality weights W , not the arbitrary input V , are separable. Since $\sum_r a_r = 1$, the row factor remains nonzero even when $\kappa = 0$.

Bounded integer-lifting and grand-product phase. Using the canonical lifts from Section 2.7, define for every column c the exact fold

$$\hat{\mu}_c := \sum_{r < 2^{d_r}} \gamma_r H_{\mathbb{Z}}[r, c] \in [0, 2^{127}). \quad (12)$$

The configured precondition $2^{d_r} q < 2^{127}$ implies this bound because $0 \leq \gamma_r < q$ and $H_{\mathbb{Z}}[r, c] \in \{0, 1\}$. The prover supplies candidates μ_c and ν_c . The verifier checks $\mu_c \in [0, 2^{127})$, $\nu_c = g^{\mu_c}$, and the reconstruction

$$\tau = \sum_c a'_c \pi_q(\mu_c). \quad (13)$$

Since H is binary, the exact fold also satisfies

$$\widehat{g^{\mu_c}} = \prod_r \left(1 + (g^{\gamma_r} - 1)H_{\mathbb{K}}[r, c]\right). \quad (14)$$

The successor PESAT relation equates this product with ν_c . Hence $\widehat{g^{\mu_c}} = g^{\mu_c}$; because both exponents lie in $[0, 2^{127})$ and $\text{ord}(g) = 2^{128} - 1$, injectivity gives $\widehat{\mu_c} = \mu_c$. Together with the input sumcheck and global reconstruction, this establishes

$$\mathcal{R}_{\text{LinBitsDual}}[h] \xRightarrow{\Pi_1} \mathcal{R}_{\text{PESAT}}[h]. \quad (15)$$

3.3 The F2Z PESAT relation

Set $y_r := g^{\gamma_r}$. Following the paper, define the F2Z grand-product polynomial for every column c by

$$Q_{\text{BitZ}, c}(\mathbf{y}, \boldsymbol{\nu}, H_{\mathbb{K}}) := \prod_r (1 + (y_r - 1)H_{\mathbb{K}}[r, c]) - \nu_c. \quad (16)$$

The underlying paper-style PESAT relation is

$$\begin{aligned} &\text{PESAT}(\mathbb{K}, Q_{\text{BitZ}}) \\ &:= \left\{ ((\mathbf{y}, \boldsymbol{\nu}), \perp; H_{\mathbb{K}}) : \mathbf{y} \in \mathbb{K}^{2^{d_r}}, \quad \boldsymbol{\nu} \in \mathbb{K}^{2^{d_c}}, \quad H_{\mathbb{K}} \in \{0_{\mathbb{K}}, 1_{\mathbb{K}}\}^{2^{d_r} \times 2^{d_c}}, \quad Q_{\text{BitZ}, c}(\mathbf{y}, \boldsymbol{\nu}, H_{\mathbb{K}}) = 0 \text{ for every } c \right\}. \end{aligned} \quad (17)$$

F2Z carries the virtualized commitment invariant alongside this polynomial relation:

$$\begin{aligned} \mathcal{R}_{\text{PESAT}}[h] := \left\{ ((\mathbf{y}, \boldsymbol{\nu}, \mathbf{M}), O; \mathbf{f}) : ((\mathbf{y}, \boldsymbol{\nu}), \perp; H_{\mathbb{K}}) \in \text{PESAT}(\mathbb{K}, Q_{\text{BitZ}}), \right. \\ \left. \begin{aligned} &\text{vec}(H) = \mathbf{M}\pi_2(\mathbf{f}), \\ &\text{Equations (1) and (2) hold} \end{aligned} \right\}. \end{aligned} \quad (18)$$

This is the virtualized lifting of the paper's F2Z PESAT relation, not a PIOP relation. Its public input is derived inside Π_1 , and its witness remains $\mathbf{f}$; its polynomial constraints are evaluated on the derived virtual view $H_{\mathbb{K}}$. The shared binding in Equations (1) and (2) ties that view to $\mathbf{f}$. Unlike a standard encoded PESAT relation, its oracle still encodes F , not H ; this is why the following reduction never queries O .

Direct reduction: layered grand-product GKR (Π_{GKR}). Define the input layer

$$L_{d_r}[r, c] := 1 + (y_r - 1)H_{\mathbb{K}}[r, c] \quad (19)$$

and recursively multiply adjacent row entries:

$$L_i[\mathbf{b}, c] := L_{i+1}[\mathbf{b}, 0, c]L_{i+1}[\mathbf{b}, 1, c], \quad \mathbf{b} \in \{0, 1\}^i, \quad 0 \leq i < d_r. \quad (20)$$

Thus $L_0[c] = \nu_c$. GKR verifies these multiplication layers from the claimed outputs back to the input layer. Its verifier returns public weights ω_{ξ} and a target σ , where $\omega_{\xi} \in \mathbb{K}^{m_h}$ and $\sigma \in \mathbb{K}$:

$$(\text{tr}', \omega_{\xi}, \sigma) \leftarrow \text{GKR.Verify}(\text{pp}_{\text{GKR}}, \mathcal{I}_{\text{GKR}}, \pi_{\text{GKR}}; \text{tr}). \quad (21)$$

The resulting terminal claim is a weighted sum of entries of $H_{\mathbb{K}}$:

$$\sum_{z \in \{0, 1\}^{m_h}} \omega_{\xi, z} H_{\mathbb{K}}[z] = \sigma. \quad (22)$$

Thus GKR implements the direct reduction

$$\mathcal{R}_{\text{PESAT}}[h] \xRightarrow{\Pi_{\text{GKR}}} \mathcal{R}_{\text{sum}}^h. \quad (23)$$

Both values are verifier-derived from the GKR transcript; all terminal contributions that do not multiply an entry of H are moved into σ . GKR reads $H_{\mathbb{K}}$ through its circuit oracle but neither queries O nor changes the commitment invariant.

3.4 The weighted-sum relation

The weighted-sum relation records the terminal linear claim produced by GKR:

$$\begin{aligned} \mathcal{R}_{\text{sum}}^h := \left\{ ((\omega_{\xi}, \sigma, \mathbf{M}), O; \mathbf{f}) : \right. \\ \left. \begin{aligned} &\sum_{z \in \{0, 1\}^{m_h}} \omega_{\xi, z} H_{\mathbb{K}}[z] = \sigma, \\ &\text{vec}(H) = \mathbf{M}\pi_2(\mathbf{f}), \\ &\text{Equations (1) and (2) hold} \end{aligned} \right\}. \end{aligned} \quad (24)$$

Direct reduction: row LinCheck (Π_{MLE}). The row protocol LinCheck_H folds the weighted claim into one evaluation point $\mathbf{r}_h \in \mathbb{K}^{m_h}$ and one claimed value $\eta \in \mathbb{K}$. Its immediate successor is

$$\mathcal{R}_{\text{sum}}^h \xRightarrow{\Pi_{\text{MLE}}} \mathcal{R}_{\text{eval}}^h. \quad (25)$$

The proof fragment for this reduction is denoted π_{MLE} . The fixed GKR wire record carries $(\pi_{\text{GKR}}, \pi_{\text{MLE}})$ in that order; the semantic split introduces no new transcript event.

3.5 The MLE-evaluation relation

The MLE-evaluation relation is

$$\begin{aligned} \mathcal{R}_{\text{eval}}^h := \left\{ ((\mathbf{r}_h, \eta, \mathbf{M}), O; \mathbf{f}) : \right. \\ \left. \begin{aligned} &\widetilde{H}_{\mathbb{K}}(\mathbf{r}_h) = \eta, \\ &\text{vec}(H) = \mathbf{M}\pi_2(\mathbf{f}), \\ &\text{Equations (1) and (2) hold} \end{aligned} \right\}. \end{aligned} \quad (26)$$

Using the $\mathbb{K}$ -valued equality polynomial from Equation (10), the MLE claim can be written as

$$\widetilde{H}_{\mathbb{K}}(\mathbf{r}_h) = \sum_{z \in \{0, 1\}^{m_h}} \text{eq}(z, \mathbf{r}_h) H_{\mathbb{K}}[z] = \eta. \quad (27)$$

Direct reduction: adjoint pullback through $M_{\mathbb{K}}^{\text{T}}$. Set

$$\mathbf{a}[z] := \text{eq}(z, \mathbf{r}_h). \quad (28)$$

Since $H_{\mathbb{K}} = M_{\mathbb{K}} \mathbf{f}_{\mathbb{K}}$, the MLE claim is rewritten directly as

$$\eta = \langle \mathbf{a}, H_{\mathbb{K}} \rangle_{\mathbb{K}} = \langle \mathbf{a}, M_{\mathbb{K}} \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}} = \left\langle M_{\mathbb{K}}^{\text{T}} \mathbf{a}, \mathbf{f}_{\mathbb{K}} \right\rangle_{\mathbb{K}}. \quad (29)$$

Set $\mathbf{a}_f := M_{\mathbb{K}}^{\text{T}} \mathbf{a}$. This is a deterministic, zero-message reduction whose immediate successor is

$$\mathcal{R}_{\text{eval}}^h \equiv_{M_{\mathbb{K}}^{\text{T}}} \mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}). \quad (30)$$

3.6 The commitment-facing linear relation

The commitment-facing relation is the encoded linear relation named $\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack})$ in the F2Z construction:

$$\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}) := \left\{ ((\mathbf{a}_f, \eta), O; \mathbf{f}) : \mathbf{a}_f \in \mathbb{K}^{n_f}, \quad \langle \mathbf{a}_f, \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}} = \eta, \right. \\ \left. O = \text{Enc}_{\mathcal{C}}(\text{Pack}(F)), \quad \text{vec}(F) = \pi_2(\mathbf{f}) \right\}. \quad (31)$$

The adjoint produces the instance $((\mathbf{a}_f, \eta), O; \mathbf{f})$ of this relation. Section 4 applies the structured ring switch directly to this claim using the canonical decomposition from Section 2.5, then invokes the recursive Ligerito opening. No post-adjoint LinCheck is used, and no reduction from $\mathcal{R}_{\text{Lin}}$ to the trivial relation is displayed in this section.

Complete direct-reduction graph. Collecting the immediate successors defined above gives

$$\mathcal{R}_{\text{LinBitsDual}}[\mathbf{h}] \xrightarrow{\Pi_1} \mathcal{R}_{\text{PESAT}}[\mathbf{h}] \xrightarrow{\Pi_{\text{GKR}}} \mathcal{R}_{\text{sum}}^h \\ \xrightarrow{\Pi_{\text{MLE}}} \mathcal{R}_{\text{eval}}^h \equiv_{\mathcal{M}_{\mathbb{K}}^T} \mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}). \quad (32)$$

4 Prover–verifier interaction

4.1 End-to-end proof decomposition and transcript channels

The end-to-end proof consists of the configured PIOP proof followed by the F2Z proof:

$$\pi_{\text{e2e}} := (\pi_{\text{PIOP}}, \pi_{\text{F2Z}}). \quad (33)$$

Verification of π_{PIOP} reduces the initial constraint statement to the F2Z handoff

$$\langle \mathbf{v}, \text{vec}(H_q) \rangle_{\mathbb{F}_q} = \mu. \quad (34)$$

The pair $(\mathbf{v}, \mu)$ is produced by the PIOP verifier and is not repeated as an F2Z proof field.

Within F2Z, write

$$\pi_1 := (\pi_{1,\text{in}}, \boldsymbol{\nu}, \boldsymbol{\mu}), \quad \pi_{1,\text{in}} := (g_i^{\text{shape}})_{i=0}^{m_h-1}. \quad (35)$$

The semantic F2Z proof object is

$$\pi_{\text{F2Z}} = (\pi_1, \pi_{\text{GKR}}, \pi_{\text{MLE}}, \mathbf{s}, \pi_{\text{open}}). \quad (36)$$

The fragment $\pi_{1,\text{in}}$ contains the round polynomials of the standard input Sumcheck that reduces Equation (34) to

$$\kappa \tilde{H}_q(\mathbf{s}_h) = \tau. \quad (37)$$

The point $\mathbf{s}_h$, scalars κ and τ , and factors $(\mathbf{a}, \mathbf{a}')$ are verifier-derived and are not additional proof fields. The ring-switch component is $\mathbf{s} = (\mathbf{s}_\theta)_{\theta \in \Theta}$, with each $\mathbf{s}_\theta \in \mathbb{K}^{128}$; the ordered set Θ and its coefficient decomposition are public and are not proof fields.

Equations (33)–(36) describe semantic proof components. The developer configuration fixes the encoding of π_{PIOP} , while Section 6 fixes the encoding of π_{F2Z} . The F2Z v1 container partitions its transmitted records into the NARG stream N_{F2Z} and the hint stream H_{F2Z} :

$$\pi_{\text{wire}} := (N_{\text{F2Z}}, H_{\text{F2Z}}), \quad \begin{aligned} \text{narglen}(\pi_{\text{F2Z}}) &:= |N_{\text{F2Z}}|, & \text{hintlen}(\pi_{\text{F2Z}}) &:= |H_{\text{F2Z}}|, \\ \text{payloadlen}(\pi_{\text{F2Z}}) &:= |N_{\text{F2Z}}| + |H_{\text{F2Z}}|, & \text{wirelen}(\pi_{\text{F2Z}}) &:= |\text{HostEncode}_{\text{v1}}(N_{\text{F2Z}}, H_{\text{F2Z}})|. \end{aligned} \quad (38)$$

Equation (38) defines the two F2Z proof streams; the following badges identify how each transmitted value is handled.

Badge	Source	Verifier action
N[x]	NARG stream	Receive x and verify it, and absorb it into the verifier's transcript.
H[x]	Hint stream	Receive x and verify it, but do not absorb it into the verifier's transcript.
D[x]	No proof bytes	Receive nothing; x is public or derived by the verifier.
N/H[π]	Both streams	Receive each field from its specified stream. Absorb NARG fields and verify Hint fields.

The verifier completes every required check and absorption before deriving the next challenge.

Transcript absorption. Every absorbed value uses the encoding

$$\text{Absorb}(\text{DOMAIN} \parallel \text{PAYLOAD_SIZE} \parallel \text{PAYLOAD}). \quad (39)$$

Section 6 specifies the canonical encoding of each domain and payload, together with the transcript replay rules. The domain identifies the semantic meaning of the absorbed value.

Right ledgers contain post-step claims; *internal* claims are shown only when they explain why a later acceptance check cannot yet discharge the original relation.

P1/V1. Bind the constraint statement and commitment

Prover P	Traffic	Verifier V	REDUCTION LEDGER
Construct the committed and PIOP-facing witness views: $\pi_2(\mathbf{h}) = \mathcal{M}\pi_2(\mathbf{f})$, $\text{vec}(H) = \pi_2(\mathbf{h})$, $\text{vec}(F) = \pi_2(\mathbf{f})$, $O := \text{Enc}_{\mathcal{C}}(P)$, retain $(\mathbf{h}, H, F, P, O)$.	$\xrightarrow{D[x, \rho]}$	Validate the constraint statement, the developer configuration, and the commitment root. Bind (x, ρ) as required by the configured PIOP.	CURRENT STATE $\mathbf{CS}[\mathbf{h}] \text{ RELATION NEW}$ $(x; H_q) \in \mathcal{R}_{\text{CS}}$ WHERE $\rho = \text{Root}(O)$

Configured matrix indexing. The developer configuration fixes a bijection between each Boolean address $z \in \{0, 1\}^{m_h}$ and a pair

$$z = (r, c) \in \{0, 1\}^{d_r} \times \{0, 1\}^{d_c}, \quad m_h = d_r + d_c.$$

Using this fixed address split, define

$$H[r, c] := [\pi_2(\mathbf{h})]_z, \quad \text{vec}(H) = \pi_2(\mathbf{h}).$$

Thus H is a matrix view of the same witness bits, not a new witness. The tables $H_{\mathbb{Z}}$, H_q , and $H_{\mathbb{K}}$ contain these same entries embedded into $\mathbb{Z}$, $\mathbb{F}_q$, and $\mathbb{K}$, respectively. We use brackets for table entries, such as $H_q[r, c]$, and parentheses only for multilinear-extension evaluations, such as $\tilde{H}_q(\mathbf{x}_r, \mathbf{x}_c)$.

P2/V2. Run the PIOP

Prover P	Traffic	Verifier V
Run the configured PIOP on the constraint statement and witness: $\pi_{\text{PIOP}} \leftarrow \Pi_{\text{PIOP}}.\text{Prove}(\text{pp}_{\text{PIOP}}, x, \text{vec}(H_q)).$	$\xrightarrow{\pi_{\text{PIOP}}}$	Run the matching PIOP verifier: $(\text{ok}_{\text{PIOP}}, v, \mu) \leftarrow \Pi_{\text{PIOP}}.\text{Verify}(\text{pp}_{\text{PIOP}}, x, \pi_{\text{PIOP}}).$ Require $\text{ok}_{\text{PIOP}} = 1$. Its output is the F2Z handoff
	$\xleftarrow{D[v, \mu]}$	$\left\langle v, \text{vec}(H_q) \right\rangle_{\mathbb{F}_q} = \mu.$

P3/V3. Reduce the PIOP handoff with Sumcheck

Prover P	Traffic	Verifier V
Run the standard m_h -round Sumcheck on the Lin-BitsDual claim from Section 2.6: $\mathbf{X} = (\mathbf{X}_r, \mathbf{X}_c),$ $G_{\text{shape}}(\mathbf{X}) := \tilde{v}(\mathbf{X}_r, \mathbf{X}_c) \cdot H_q(\mathbf{X}_r, \mathbf{X}_c),$ $\mu = \sum_{r,c} G_{\text{shape}}(r, c),$ $\pi_{1,\text{in}} := (g_i^{\text{shape}})_{i=0}^{m_h-1}.$	$\xrightarrow{N[\pi_{1,\text{in}}]}$	Initialize the F2Z transcript by binding its domain, handoff statement, configuration, and commitment root. Verify the Sumcheck transcript. It yields a terminal point $\mathbf{s}_h = (s_r, s_c)$ and value τ satisfying $\tau = \tilde{v}(s_r, s_c) \tilde{H}_q(s_r, s_c), \quad \kappa := \tilde{v}(s_r, s_c),$ $a_r := \text{eq}_{\mathbb{F}_q}(r, s_r),$ and derive $a'_c := \kappa \text{eq}_{\mathbb{F}_q}(c, s_c).$ The resulting terminal claim is $\kappa \tilde{H}_q(s_r, s_c) = \tau = \sum_c a'_c \sum_r a_r H_q[r, c].$ Absorb the derived seam and derive the canonical integer lifts $\gamma_r := \text{lift}_q(a_r).$
	$\xleftarrow{D[s_h, \kappa, \tau]}$	

P4/V4. Fold each column and encode the folds as product claims

Prover P	Traffic	Verifier V
For every column c , compute $\hat{\mu}_c := \sum_r \gamma_r H_{\mathbb{Z}}[r, c],$ $\mu_c := \hat{\mu}_c,$ $\nu_c := g^{\mu_c} \in \mathbb{K}^{\times}.$	$\xrightarrow{N[\nu]}$ $\xrightarrow{H[\mu]}$	Before deriving any dependent challenge, require $ \mu = 2^{d_c},$ and, for every $c,$ $0 \leq \mu_c < 2^{127}, \quad g^{\mu_c} = \nu_c.$ Then close the retained Sumcheck claim by checking $\sum_c a'_c \pi_q(\mu_c) = \tau.$
	$\xleftarrow{D[\zeta]}$	After accepting the hint, derive $\zeta \leftarrow \text{Squeeze}_{\mathbb{K}}^{d_c}(\text{tr}),$ $e_0 := \tilde{v}(\zeta).$ Record the successor claim for P5/V5: $Q_{r,c} := 1 + (g^{\gamma_r} - 1) H_{\mathbb{K}}[r, c],$ $\nu_c = \prod_r Q_{r,c}.$ This equality is not checked in P4/V4, and no entry of $H_{\mathbb{K}}$ is sent.

REDUCTION LEDGER

PIOP Π_{PIOP}
FROM $\mathcal{R}_{\text{CS}}$
TO $\mathcal{R}_{\text{LinBitsDual}}[\mathbf{h}]$

CURRENT STATE

LIN $[\mathbf{h}]$ **RELATION** **NEW**
 $\langle v, \text{vec}(H_q) \rangle_{\mathbb{F}_q} = \mu$

WHERE

$\rho \quad \rho = \text{Root}(O)$

REDUCTION LEDGER

SUMCHECK $\Pi_{1,\text{in}}$
FROM $\mathcal{R}_{\text{LinBitsDual}}[\mathbf{h}]$
TO $\kappa \tilde{H}_q(s_h) = \tau$

CURRENT STATE

SC $_{\text{in}}[\mathbf{H}]$ **INTERNAL** **NEW**
 $\kappa \tilde{H}_q(s_h) = \tau$

WHERE

$\rho \quad \rho = \text{Root}(O)$

REDUCTION LEDGER

CURRENT STATE

PESAT **RELATION** **NEW**
 $\mathcal{R}_{\text{PESAT}}[\mathbf{h}]$
 $\nu_c = \prod_r (1 + (g^{\gamma_r} - 1) H_{\mathbb{K}}[r, c])$

WHERE

$\rho \quad \rho = \text{Root}(O)$
 $P \quad P = \text{Pack}(F)$

P5/V5. Run the GKR reduction

Prover P	Traffic	Verifier V
Set $\mathcal{I}_{\text{GKR}} := \text{GKRInput}(\gamma, \nu, \zeta, e_0)$ and compute $\pi_{\text{GKR}} \leftarrow \text{GKR.Prove}(\text{pp}_{\text{GKR}}, \mathcal{I}_{\text{GKR}}, H_{\mathbb{K}}; \text{tr})$.	$\xrightarrow{\text{N/H}[\pi_{\text{GKR}}]}$	Compute $(\text{tr}', \omega_{\xi}, \sigma) \leftarrow \text{GKR.Verify}(\text{pp}_{\text{GKR}}, \mathcal{I}_{\text{GKR}}, \pi_{\text{GKR}}; \text{tr})$, require success and the exact subordinate endpoint, then set $\text{tr} \leftarrow \text{tr}'$. The successor is $\sum_{z \in \{0,1\}^{m_h}} \omega_{\xi,z} H_{\mathbb{K}}[z] = \sigma.$

GKR operates on $H_{\mathbb{K}}$ and does not query O .

P6/V6. Reduce the GKR sum claim to an MLE claim

Prover P	Traffic	Verifier V
Run the standard m_h -round Sumcheck on the weighted-sum claim: $G_{\text{MLE}}(\mathbf{X}) := \tilde{\omega}_{\xi}(\mathbf{X}) \tilde{H}_{\mathbb{K}}(\mathbf{X}),$ $\sigma = \sum_{z \in \{0,1\}^{m_h}} G_{\text{MLE}}(z),$ $\pi_{\text{MLE}} := \left((g_i^{\text{MLE}})_{i=0}^{m_h-1}, \eta \right),$ where the prover supplies $\eta := \tilde{H}_{\mathbb{K}}(\mathbf{r}_h)$ at the transcript-derived terminal point $\mathbf{r}_h$.	$\xrightarrow{\text{N/H}[\pi_{\text{MLE}}]}$	Verify the Sumcheck transcript. It yields a terminal point $\mathbf{r}_h$ and value θ . Compute $\lambda := \tilde{\omega}_{\xi}(\mathbf{r}_h)$ and require the terminal check $\theta = \lambda \eta.$ The resulting successor claim is $\tilde{H}_{\mathbb{K}}(\mathbf{r}_h) = \eta.$

The row LinCheck operates on $H_{\mathbb{K}}$ and does not query O .

P7/V7. Rewrite $\langle \mathbf{a}, M\mathbf{f} \rangle$ as $\langle M^{\top} \mathbf{a}, \mathbf{f} \rangle$

Prover P	Traffic	Verifier V
Both parties set $\mathbf{a}[z] := \text{eq}(z, \mathbf{r}_h)$ and use $H_{\mathbb{K}} = M_{\mathbb{K}} \mathbf{f}_{\mathbb{K}}$ to rewrite $\eta = \langle \mathbf{a}, H_{\mathbb{K}} \rangle_{\mathbb{K}} = \langle \mathbf{a}, M_{\mathbb{K}} \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}} = \langle M_{\mathbb{K}}^{\top} \mathbf{a}, \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}}.$ Set $\mathbf{a}_f := M_{\mathbb{K}}^{\top} \mathbf{a}$. No message or challenge is needed. In the configured packing order, both parties also view this same coefficient as $A_f[v, j] := \mathbf{a}_f[128j + v].$ This is only a change of coordinates; the claim remains $\langle \mathbf{a}_f, \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}} = \eta$.		

REDUCTION LEDGERGKR SUBPROTOCOL Π_{GKR}

FROM $\mathcal{R}_{\text{PESAT}}^h[h]$
TO $\mathcal{R}_{\text{sum}}^h$

CURRENT STATE

SUM[H] RELATION NEW

$$S(H_{\mathbb{K}}) = \sigma$$

$$S(H_{\mathbb{K}}) = \sum_z \omega_{\xi,z} H_{\mathbb{K}}[z]$$

WHERE

$\rho \quad \rho = \text{Root}(O)$
 $P \quad P = \text{Pack}(F)$

REDUCTION LEDGERROW LINCHECK Π_{MLE}

FROM $\mathcal{R}_{\text{sum}}^h$
TO $\mathcal{R}_{\text{eval}}^h$

CURRENT STATE

EVAL[H] RELATION NEW

$$\tilde{H}_{\mathbb{K}}(\mathbf{r}_h) = \eta$$

WHERE

$\rho \quad \rho = \text{Root}(O)$

REDUCTION LEDGER

ADJOINT IDENTITY $\langle \mathbf{a}, M_{\mathbb{K}} \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}} = \langle M_{\mathbb{K}}^{\top} \mathbf{a}, \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}}$

FROM $\mathcal{R}_{\text{Lin}}^h(\mathbb{K}, \mathcal{C}, \text{Pack})$
TO $\mathcal{R}_{\text{Lin}}^h(\mathbb{K}, \mathcal{C}, \text{Pack})$

CURRENT STATE

LIN[f] RELATION NEW

$$\langle \mathbf{a}_f, \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}} = \eta$$

WHERE

$\rho \quad \rho = \text{Root}(O)$

P8/V8. Ring-switch the adjoint claim into the committed packing

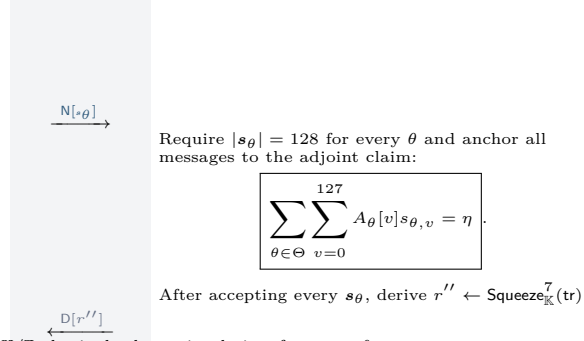
Prover P	Traffic	Verifier V
From the public coefficient A_f , both parties derive the canonical ordered decomposition		

$$A_f[v, j] = \sum_{\theta \in \Theta} A_\theta[v] B_\theta[j]$$

from Section 2.5. This represents the same inner product produced by P7/V7; it is not another reduction or proof claim.

For every $\theta \in \Theta$ and $0 \leq v < 128$, compute

$$s_{\theta, v} := \sum_{j < 2^{mp}} B_\theta[j] F_v[j].$$



```

3: [C] Require[ValidateConfig(pp,  $\mathcal{I}_{\text{LinBitsDual}}, \rho$ ) = 1]
4: Read  $(n_f, m_f, n_h, m_h, d_{F,r}, d_{F,c}, d_r, d_c, g)$  from the fixed public configuration
5: [C] Require[ $n_f = 2^{m_f} \wedge n_h = 2^{m_h} \wedge m_h = d_r + d_c \wedge 128 \mid n_f \wedge m_f = d_{F,r} + d_{F,c} \wedge m_f \geq 7 \wedge 2^{dr} q < 2^{127}$ ]
6: [D]  $m_p \leftarrow m_f - 7$ 
7: [D]  $b_{\text{virt}} \leftarrow \text{HasVirtualization}(\text{configuration})$ 
8: if  $b_{\text{virt}}$  then
9:   Read  $M$  from the developer configuration and [C] Require[ $M \in \mathbb{F}_2^{n_h \times n_f}$ ]
10: else
11:   [C] Require[ $n_h = n_f$ ] and [C] Require[SameFlatOrder(configuration) = 1]
12:   [D]  $M \leftarrow I_{n_f}$ 
13: end if
14: [C] Require[ord( $g$ ) =  $2^{128} - 1$ ]
15: [C] Require[ $|v| = n_h \wedge \mu \in \mathbb{F}_q$ ]
16: [D]  $\text{tr} \leftarrow \text{tr}_{\text{init}}$ 
17: [A] Absorb( $D_{\text{domain}}$ ; DomainV1) using Equation (52)
18: [A] Absorb( $D_{\text{statement}}$ ; StatementV1) using Equation (52)
19: Initialize independent cursors [D]  $c_N \leftarrow \text{Cursor}(N_{F2Z}, n_N, b_N)$ ,  $c_H \leftarrow \text{Cursor}(H_{F2Z}, n_H, b_H)$ , and PendingHint  $\leftarrow 0$ 
20: Load the fixed input-Sumcheck, GKR, row-LinCheck, and opening ledgers
21: [D]  $\sigma_{-1}^{\text{shape}} \leftarrow \mu$ 
22: for  $i = 0, \dots, m_h - 1$  do
23:    $(g_i^{\text{shape}}, p_i^{\text{shape}}) \leftarrow [\text{N}] \text{ReadNarg}(c_N, D_{\text{shape}}, i)$ 
24:   [C] Require[ $g_i^{\text{shape}} \in \mathbb{F}_q[X] \wedge \deg g_i^{\text{shape}} \leq 2 \wedge g_i^{\text{shape}}(0) + g_i^{\text{shape}}(1) = \sigma_{i-1}^{\text{shape}}$ ]
25:   [A] Absorb( $D_{\text{shape}}, i; p_i^{\text{shape}}$ )
26:   [D]  $(\text{tr}, s_{h,i}) \leftarrow \text{SqueezeFqV1}(\text{tr}, D_{\text{shape-challenge}}, i)$ 
27:   [D]  $\sigma_i^{\text{shape}} \leftarrow g_i^{\text{shape}}(s_{h,i})$ 
28: end for
29: [D]  $s_h \leftarrow (s_{h,0}, \dots, s_{h,m_h-1})$ ,  $\tau \leftarrow \sigma_{m_h-1}^{\text{shape}}$ ,  $\kappa \leftarrow \tilde{v}(s_h)$ 
30: Split  $s_h = (s_r, s_c)$  with  $|s_r| = d_r$  and  $|s_c| = d_c$ 
31: [D]  $a_r \leftarrow \text{eq}_{\mathbb{F}_q}(r, s_r)$  and  $a'_c \leftarrow \kappa \text{eq}_{\mathbb{F}_q}(c, s_c)$ 
32: [C] Require[ $|a| = 2^{d_r} \wedge |a'| = 2^{d_c} \wedge \sum_r a_r = 1$ ]
33: Retain the terminal claim  $\kappa \tilde{H}_g(s_h) = \tau$ 
34: [A] Absorb( $D_{\text{seam}}$ ; LinCheckSeamV1) using Equation (53)
35: For every row  $r$ , derive the unique  $\gamma_r \in [0, q)$  with  $\pi_q(\gamma_r) = a_r$ 
36: Fix, without advancing either cursor, the remaining record schedule

```

P4/V4: Fold each column and encode the folds as product claims

```

37:  $(\nu, p_\nu) \leftarrow [\text{N}] \text{ReadNarg}(c_N, D_\nu)$ 
38: [C] Require[ $|\nu| = 2^{d_c} \wedge \nu \in (\mathbb{K}^\times)^{2^{d_c}}$ ]
39: [A] Absorb( $D_\nu; p_\nu$ )
40: PendingHint  $\leftarrow 1$ ,  $(\mu, p_\mu) \leftarrow [\text{H}] \text{ReadHint}(c_H, D_\mu)$ 
41: [C] Require[ $|\mu| = 2^{d_c} \wedge \forall c: 0 \leq \mu_c < 2^{127} \wedge g^{\mu_c} = \nu_c$ ]
42: [C] Require[ $\sum_c a'_c \pi_q(\mu_c) = \tau$ ]
43: PendingHint  $\leftarrow 0$ 
44:  $(\text{tr}, \zeta) \leftarrow [\text{S}] \text{Squeeze}_{\mathbb{K}}^{d_c}(\text{tr}; D_\zeta)$ ,  $e_0 \leftarrow \tilde{v}(\zeta)$ 

```

P5/V5: Reduce PESAT to a weighted sum with GKR

$$\mathcal{R}_{\text{PESAT}}[h] \xrightarrow{\Pi_{\text{GKR}}} \mathcal{R}_{\text{sum}}^h$$

```

45: [D]  $\mathcal{I}_{\text{GKR}} \leftarrow \text{GKRInput}(\gamma, \nu, \zeta, e_0)$ 
46:  $(\text{ok}_{\text{GKR}}, \text{tr}', \omega_\xi, \sigma) \leftarrow [\text{CALL}] \text{GKR.Verify}(\text{pp}_{\text{GKR}}, \mathcal{I}_{\text{GKR}}; \text{tr}, c_N, c_H)$ 
47: [C] Require[okGKR = 1]
48: [D]  $\text{tr} \leftarrow \text{tr}'$ 

```

P6/V6: Reduce the weighted sum to an MLE claim

$$\mathcal{R}_{\text{sum}}^h \xrightarrow{\Pi_{\text{MLE}}} \mathcal{R}_{\text{eval}}^h$$

```

49: [D]  $\mathcal{I}_{\text{MLE}} \leftarrow (\omega_\xi, \sigma, \text{configuration})$ 
50:  $(\text{ok}_{\text{MLE}}, \text{tr}', r_h, \theta, \eta) \leftarrow [\text{CALL}] \text{RowLinCheck.Verify}(\mathcal{I}_{\text{MLE}}; \text{tr}, c_N, c_H)$ 
51: [C] Require[okMLE = 1] and [D]  $\text{tr} \leftarrow \text{tr}'$ 
52: [D]  $\lambda \leftarrow \tilde{\omega}_\xi(r_h)$ 
53: [C] Require[ $\theta = \lambda \eta$ ]

```

No absorb or squeeze occurs at the semantic P5/V5–P6/V6 boundary; the fixed record contains $(\pi_{\text{GKR}}, \pi_{\text{MLE}})$ in that order.

P7/V7: Rewrite $\langle a, Mf \rangle$ through the adjoint

```

54: [D]  $a[z] \leftarrow \text{eq}(z, r_h)$ 
 $\langle a, H_{\mathbb{K}} \rangle_{\mathbb{K}} = \langle a, M_{\mathbb{K}} f_{\mathbb{K}} \rangle_{\mathbb{K}} = \langle M_{\mathbb{K}}^T a, f_{\mathbb{K}} \rangle_{\mathbb{K}}$ 

```

```

55: [D]  $a_f \leftarrow M_{\mathbb{K}}^T a$ 
56: [D]  $A_f[v, j] \leftarrow a_f[128j + v]$ 

```

P8/V8: Ring-switch the adjoint claim into the committed packing

```

57: [D]  $(\Theta, (A_\theta, B_\theta)_{\theta \in \Theta}) \leftarrow [\text{CALL}] \text{RingDecomp}(a_f)$ 
58: [C] Require[ $\Theta \neq \emptyset \wedge \forall v, j: A_f[v, j] = \sum_{\theta \in \Theta} A_\theta[v] B_\theta[j]$ ]
59: for  $\theta \in \Theta$  in canonical order do
60:    $(s_\theta, p_{s,\theta}) \leftarrow [\text{N}] \text{ReadNarg}(c_N, D_{s,\theta})$ 
61:   [C] Require[ $|s_\theta| = 128$ ]
62:   [A] Absorb( $D_{s,\theta}; p_{s,\theta}$ )
63: end for
64: [C] Require[ $\sum_{\theta \in \Theta} \sum_{v=0}^{127} A_\theta[v] s_{\theta,v} = \eta$ ]
65:  $(\text{tr}, r'') \leftarrow [\text{S}] \text{Squeeze}_{\mathbb{K}}^{\text{tr}}(\text{tr}; D_{r''})$ 
66: [D]  $\Phi_{r''}(x) \leftarrow \sum_u \text{eq}(u, r'')[x]_u$ 
67: for  $\theta \in \Theta$  in canonical order do
68:   [D]  $t_{\theta,u} \leftarrow \sum_v [s_{\theta,v}]_u \beta_v$ 
69:   [D]  $\beta_\theta \leftarrow \sum_u \text{eq}(u, r'') t_{\theta,u}$ ,  $\omega_\theta[j] \leftarrow \Phi_{r''}(B_\theta[j])$ 
70:    $(\text{tr}, \lambda_\theta) \leftarrow [\text{S}] \text{Squeeze}_{\mathbb{K}}(\text{tr}; D_{\lambda,\theta})$ 
71: end for
72: [D]  $B \leftarrow \sum_{\theta \in \Theta} \lambda_\theta \omega_\theta$ ,  $\beta \leftarrow \sum_{\theta \in \Theta} \lambda_\theta \beta_\theta$ 

```

P9/V9: Open the packed claim

```

73: [D]  $p_{\text{target}} \leftarrow \text{LE}_{32}(m_p) \parallel \text{Enc}_{\mathbb{K}}(\beta)$ 
74: [A]  $\text{Absorb}(D_{\text{target}}; p_{\text{target}})$ 
75: [C] Require[[CALL]  $\text{RecVerify}(\text{pp}, \rho, B, \beta; \text{tr}, c_N, c_H) = 1$ ]
76: [C] Require[NoPendingHint = 1  $\wedge$  NargEOF( $c_N$ ) = 1  $\wedge$  HintEOF( $c_H$ ) = 1  $\wedge$  HostEOF( $\pi_{\text{host}}$ ) = 1  $\wedge$  Counts( $c_N, c_H$ ) = ( $n_N, n_H$ )]
77: return true

```

The helper verifiers own their subordinate ledgers. Every NARG read advances only c_N , every hint read advances only c_H , and no helper returns with an unchecked hint. P4/V4's $\text{N}[\nu]$, $\text{H}[\mu]$ order still uses the NARG-then-Hint order fixed in Section 4. The byte rules in Section 6 govern every read, absorb, squeeze, PoW check, and EOF condition.

6 Wire and transcript format v1 (Optional)

This optional section defines the concrete v1 profile. Optionality concerns whether an implementation selects this profile; once selected, every rule in this section is mandatory. The section fixes the bytes consumed by a conforming v1 prover and verifier. The proof system has two physical, canonically encoded positional record streams, a BLAKE3 transcript, SHA-256 Merkle roots, and the recursive Ligerito and proof-of-work parameters below. A proof using the legacy positional/bincode codec is not a v1 proof.

6.1 Version and fixed algorithms

The eight-byte container magic is the ASCII string `F2ZPCS\0\0`, and the wire version is one. This document defines the not-yet-frozen v1 format; its input-shaping ledger supersedes the earlier draft that reserved the entire 1000-1fff range. Once a complete v1 proof/transcript fixture is published, changing any normative rule in this section requires a new wire version.

The transcript and its XOF are unkeyed BLAKE3; Merkle nodes and roots use SHA-256; PoW uses unkeyed BLAKE3-256. The field basis is the polynomial basis fixed in Section 2.1. The public parameters fix the GKR protocol, constrained-code construction, transcript, and opening protocol. None is selected or negotiated by proof bytes. A proof system without a byte-exact GKR ledger or without a canonical codeword construction and root vectors is incomplete and MUST NOT accept proofs.

Write $m := m_f$ and $m_p := m - 7$ for the packed-vector dimension. For the deployed range $22 \leq m \leq 35$, let J_m be the number of recursive opening levels. The level shapes and all parameters except the base fold difficulties are fixed by

$$\begin{aligned}
h_i &:= m - 11 - 3i, & J_m &:= \min\{J \geq 1 : h_{J-1} \in \{3, 4, 5\}\}, \\
r_i &:= 1 + i, & k_i &:= \begin{cases} 4, & i = 0, \\ 3, & i > 0, \end{cases} \\
Q_i &:= (183, 90, 60, 45, 36, 31, 27, 24)_i, & g_i &:= 16, \\
o_i &:= \begin{cases} 0, & i = 0, \\ 1, & i > 0. \end{cases}
\end{aligned} \tag{40}$$

Here h_i is the message-column exponent, r_i the inverse-rate exponent, k_i the fold count, Q_i the query count, g_i the query-grinding difficulty, and o_i the OOD sample count. Implementations MUST use the integers in Equation (40), not a floating-point recomputation.

The remaining base fold difficulties f_i are the following literal table. Only the first J_m entries of the row for m exist.

m	$(f_0, \dots, f_{J_m-1})$
22	(9, 6, 3)
23	(10, 7, 4, 1)
24	(11, 8, 5, 2)
25	(12, 9, 6, 3)
26	(13, 10, 7, 4, 2)
27	(14, 11, 8, 5, 3)
28	(15, 12, 9, 6, 4)
29	(16, 13, 10, 7, 5, 3)
30	(17, 14, 11, 8, 6, 4)
31	(18, 15, 12, 9, 7, 5)
32	(19, 16, 13, 10, 8, 6, 3)
33	(20, 17, 14, 11, 9, 7, 4)
34	(21, 18, 15, 12, 10, 8, 5)
35	(22, 19, 16, 13, 11, 9, 6, 6)

6.2 Primitive canonical encodings

All unsigned integers are fixed-width little-endian. Decoders MUST NOT reduce, truncate, ignore high bits, or convert to a host-sized integer before checking that the value fits. The following encodings are the only canonical ones.

Type	Bytes	Canonicity rule
$u_8, u_{16}, u_{32}, u_{64}, u_{128}$	1, 2, 4, 8, 16	Unsigned little-endian of exactly the named width.
$a \in \mathbb{K}$	16	$\text{LE}_{64}(a_{\text{lo}}) \parallel \text{LE}_{64}(a_{\text{hi}})$ in the fixed polynomial basis; coefficient bit j is bit j of the resulting 128-bit word.
$a \in \mathbb{F}_q$	16	$\text{LE}_{128}(\bar{a})$, where $0 \leq \bar{a} < q$ and $\pi_q(\bar{a}) = a$; reject an integer at least $q = 2^{100} - 15$.
Bounded exponent u	16	$\text{LE}_{128}(u)$; require $u < 2^{127}$.
Nonce	8	$\text{LE}_{64}(n)$.
Merkle value	32	Raw SHA-256 digest bytes, in digest output order.
Vector $[x_0, \dots, x_{n-1}]$	4 plus elements	$\text{LE}_{32}(n)$ followed by canonical elements in index order; n must equal the protocol-derived expected count.
Opened rows and path	nested	Vector of rows, each a vector of $\mathbb{K}$ elements, followed by a vector of 32-byte Merkle values. Expected row, width, and path counts are derived before allocation.

Every composite is concatenation without padding. Options, Rust enums, **usize**, varints, serde defaults, and bincode layouts are not wire types. For every accepted primitive or composite x , decoding its encoding must consume the whole bounded slice and re-encoding the result must reproduce the same bytes.

6.3 Merkle rows and canonical multiproofs

For opening level $0 \leq k < J_m$, define the message dimension, tree depth, leaf count, and row width by

$$n_k := h_k + k_k, \quad D_k := h_k + r_k, \quad \Lambda_k := 2^{D_k}, \quad I_k := 2^{k_k}. \tag{41}$$

Equation (41) implies $n_0 = m - 7$, $n_k = h_{k-1}$ for $k > 0$, $D_k = m - 10 - 2k$, $I_0 = 16$, and $I_k = 8$ for $k > 0$. Let $C_k \in \mathbb{K}^{\Lambda_k \times I_k}$ be the level- k codeword matrix, with leaves and lanes in increasing integer order. Its exact row and tree bytes are

$$\begin{aligned} \text{RowBytes}_k(q) &:= \text{Enc}_{\mathbb{K}}(C_k[q, 0]) \parallel \dots \parallel \text{Enc}_{\mathbb{K}}(C_k[q, I_k - 1]), \\ H_{k,0,q} &:= \text{SHA256}(\text{RowBytes}_k(q)), \\ H_{k,d+1,j} &:= \text{SHA256}(H_{k,d,2j} \parallel H_{k,d,2j+1}), \\ \text{root}_k &:= H_{k,D_k,0}. \end{aligned} \quad (42)$$

Here $\text{Enc}_{\mathbb{K}}$ is the 16-byte encoding in Section 6.2. Equation (42) uses no native-memory cast, row-length prefix, row index, level domain code, padding, or leaf/internal-node domain byte. Adding such a byte changes every root and requires a new wire version. The fixed code specification defines how C_k is generated; this section fixes its portable row and tree encoding.

Let the already-derived query set be $A_0 = (q_0 < \dots < q_{Q_k-1})$. At depth d , scan the active positions A_d in increasing order. If both siblings are active, combine the pair once and emit no digest. Otherwise emit the digest at position $p \text{ xor } 1$, using p 's parity to place it on the left or right. Then set $A_{d+1} := \text{sortuniq}\{[p/2] : p \in A_d\}$. The canonical multiproof is the concatenation of emitted digests, bottom-up and in that scan order, and its exact count is

$$M_k := \sum_{d=0}^{D_k-1} |\{p \in A_d : (p \text{ xor } 1) \notin A_d\}|. \quad (43)$$

Equation (43) fixes the exact number of transmitted path digests. The level- k opening payload is exactly

$$\text{LE}_{32}(Q_k) \parallel \prod_{j=0}^{Q_k-1} (\text{LE}_{32}(I_k) \parallel \text{RowBytes}_k(q_j)) \parallel \text{LE}_{32}(M_k) \parallel \prod_{u=0}^{M_k-1} \text{path}_k[u], \quad (44)$$

with byte length

$$8 + Q_k(4 + 16I_k) + 32M_k. \quad (45)$$

The verifier derives A_d , M_k , and Equation (45) from the sampled queries before allocating or reading the bounded Hint record, requires rows in q_j order, reconstructs Equation (42), and rejects unused path digests or payload bytes.

6.4 Outer two-stream container

Here N and H are the concatenations of their canonical payload records in the verifier-event and hint-check orders, respectively. They contain no transmitted event domain, scope, or generic record-length wrapper; each next decoder and its bounded length follow from the fixed ledger and the canonical payload type.

The 40-byte header is fixed as follows. Offsets and sizes are decimal; all multibyte integers use Section 6.2.

offset	size	field	
0	8	magic = 46 32 5a 50 43 53 00 00	
8	2	wire_version = 01 00	
10	2	header_len = 28 00	(40)
12	4	flags = 00 00 00 00	
16	4	narg_record_count	
20	4	hint_record_count	
24	8	narg_byte_len	
32	8	hint_byte_len	
40	...	NARG records, then Hint records	

Writing Hdr_{40} for those exact header bytes, the only v1 host encoding is

$$\text{HostEncode}_{v1}(N, H) := \text{Hdr}_{40} \parallel N \parallel H, \quad \text{bytelen}(\text{HostEncode}_{v1}(N, H)) = 40 + \text{bytelen}(N) + \text{bytelen}(H). \quad (46)$$

Equation (46) implies the four benchmark metrics

$$\begin{aligned} \text{narglen}(\pi) &= \text{bytelen}(N), & \text{hintlen}(\pi) &= \text{bytelen}(H), \\ \text{payloadlen}(\pi) &= \text{bytelen}(N) + \text{bytelen}(H), & \text{wirelen}(\pi) &= 40 + \text{payloadlen}(\pi). \end{aligned} \quad (47)$$

The decoder MUST verify Equation (47) with checked arithmetic before allocating, enforce statement-derived upper bounds, and require input length exactly $40 + \text{narg_byte_len} + \text{hint_byte_len}$. The two declared record counts must match the records actually consumed. Header count and length fields are transport metadata and are not transcript input.

6.5 Typed event records and domain registry

Every absorbed NARG value, public value, and squeeze event uses the three-part encoding from Equation (39). Its canonical domain is the following 16-byte string:

$$D_{\tau,j,k,i} := \text{LE}_{16}(\tau) \parallel \text{LE}_{16}(0) \parallel \text{LE}_{32}(j) \parallel \text{LE}_{32}(k) \parallel \text{LE}_{32}(i). \quad (48)$$

Here τ is the numeric semantic-domain code and (j, k, i) is its verifier-derived scope. Let $\perp_{32} := 0\text{xffffffff}$ denote an inapplicable coordinate. These values are fixed by the current ledger position; they are never read from either proof stream.

The names used by the verifier algorithm in Section 5 abbreviate

$$\begin{aligned} D_{\text{domain}} &:= D_{0\text{x}0001, \perp_{32}, \perp_{32}, \perp_{32}}, & D_{\text{statement}} &:= D_{0\text{x}0002, \perp_{32}, \perp_{32}, \perp_{32}}, \\ D_{\text{shape}, i} &:= D_{0\text{x}1002, \perp_{32}, \perp_{32}, i}, & D_{\text{shape-challenge}, i} &:= D_{0\text{x}1101, \perp_{32}, \perp_{32}, i}, \\ D_{\text{seam}} &:= D_{0\text{x}1003, \perp_{32}, \perp_{32}, \perp_{32}}, & D_{\nu} &:= D_{0\text{x}2001, \perp_{32}, \perp_{32}, \perp_{32}}, \\ D_{\mu} &:= D_{0\text{x}2081, \perp_{32}, \perp_{32}, \perp_{32}}, & D_{s, \theta} &:= D_{0\text{x}4001, \theta, \perp_{32}, \perp_{32}}, & D_{\lambda, \theta} &:= D_{0\text{x}4102, \theta, \perp_{32}, \perp_{32}}, \\ D_{\text{target}} &:= D_{0\text{x}5001, \perp_{32}, \perp_{32}, \perp_{32}}. \end{aligned} \quad (49)$$

The vector names D_{ζ} and D_{ν} denote consecutive coordinate domains $D_{0\text{x}2101, \perp_{32}, \perp_{32}, i}$ and $D_{0\text{x}4101, \perp_{32}, \perp_{32}, i}$, respectively.

For payload z , an absorbed event appends exactly

$$D_{\tau,j,k,i} \parallel \text{LE}_{64}(\text{bytelen}(z)) \parallel z. \quad (50)$$

There is no frame or event wrapper. A transmitted NARG or Hint record is only the canonical payload z . Its decoder and expected length are determined by the fixed event ledger, including any canonical length prefix belonging to the payload type itself. Thus proof parsing is positional and separate from transcript absorption: a NARG read decodes and validates z , then applies Equation (50); a Hint read decodes and checks z but never applies that equation.

The domain registry is below. Domain codes and scopes are verifier-derived transcript inputs, not record headers. **D** events are public or derived, have no proof record, and are absorbed at their ledger position; **N** payloads occur only in N and are absorbed; **H** payloads occur only in H and are checked but never absorbed; **S** domains are squeeze requests. For an **S** base domain code τ , the result domain code is τ OR $0\text{x}8000$ and is derived, never transmitted. For an **S** row, the last column describes the derived result; its request payload is always $\text{LE}_{32}(n) \parallel a$ as fixed by Equation (54).

Code	Class	Event	Scope; proof payload or squeeze result
0001	D	domain/version	$(\perp, \perp, \perp)$; DomainV1 from Equation (52).
0002	D	statement	$(\perp, \perp, \perp)$; StatementV1 from Equation (52).
0003	—	legacy core seam	Reserved and MUST NOT be scheduled; the input Sumcheck terminates with derived domain code 1003.
1001	—	PIOP row collapse	Reserved for a legacy PIOP-side phase; it does not occur when F2Z receives the public v -weighted claim.
1002	N	input-Sumcheck round	$(\perp, \perp, i)$ for $0 \leq i < m_h$; exactly $\text{Enc}_{\mathbb{F}_q}(g_i(0)) \parallel \text{Enc}_{\mathbb{F}_q}(g_i(1)) \parallel \text{Enc}_{\mathbb{F}_q}(g_i(2))$ for the degree-two round polynomial, with no vector prefix.
1003	D	input-Sumcheck seam	$(\perp, \perp, \perp)$; LinCheckSeamV1 from Equation (53).
1101	S	input-Sumcheck challenge	$(\perp, \perp, i)$ for $0 \leq i < m_h$; one $\mathbb{F}_q$ challenge from the rejection sampler below.
1000	—	reserved	The verifier MUST NOT schedule this domain code.
1004–1100	—	reserved	The verifier MUST NOT schedule these domain codes.
1102–1fff	—	reserved	The verifier MUST NOT schedule these domain codes.
2001	N	fold exponent image ν	$(\perp, \perp, \perp)$; vector of 2^{d_c} elements of $\mathbb{K}$.
2081	H	integer fold μ	$(\perp, \perp, \perp)$; vector of 2^{d_c} canonical $u_{128} < 2^{127}$.
2101	S	fold challenge ζ	$(\perp, \perp, i)$ for $0 \leq i < d_c$; one $\mathbb{K}$ challenge.
3001	N	GKR phase- c sum	$(\perp, k, \perp)$; one $\mathbb{K}$ element, absorbed after the phase-link check and before the first phase- c round.
3002–30ff	Sub.	subprotocol messages	The fixed subordinate ledger owns exact message domain codes, codecs, and checks for π_{GKR} followed by π_{MLE} .
3100–31ff	Sub.	subprotocol squeezes	The fixed subordinate ledger owns exact squeeze domain codes and result types in the same order.
4001	N	ring s_θ vector	$(\theta, \perp, \perp)$ for $\theta \in \Theta$ in canonical order; vector of exactly 128 elements of $\mathbb{K}$.
4002	—	reserved	The verifier MUST NOT schedule this domain code.
4101	S	shared r''	$(\perp, \perp, i)$ for $0 \leq i < 7$; one $\mathbb{K}$ challenge.
4102	S	ring batch coefficient λ_θ	$(\theta, \perp, \perp)$ for $\theta \in \Theta$ in canonical order; one $\mathbb{K}$ challenge.
4103	—	reserved	The verifier MUST NOT schedule this domain code.
5001	D	opening target	$(\perp, \perp, \perp)$; $\text{LE}_{32}(m_p) \parallel \text{Enc}_{\mathbb{K}}(\beta)$. B is recomputed.
5002	N	initial root	$(\perp, 0, \perp)$; one 32-byte SHA-256 digest.
5003	N	recursive root	$(\perp, k, \perp)$, $1 \leq k < J_m$; one digest.
5004	N	sumcheck message	scope by kind below; kind: u_8 followed by $\text{Enc}_{\mathbb{K}}(u_0) \parallel \text{Enc}_{\mathbb{K}}(u_2)$.
5005	N	OOD value	$(a, k, \perp)$; one $\mathbb{K}$ element for sample a .
5006	N	final y_r	$(\perp, J_m - 1, \perp)$; vector of exactly $2^h J_m - 1$ elements of $\mathbb{K}$.
5007	N	fold-PoW nonce	$(\perp, k, i)$; one u_{64} .
5008	N	query-PoW nonce	$(\perp, k, \perp)$; one u_{64} .
5080	H	initial opening	$(\perp, 0, \perp)$; Equation (44).
5081	H	recursive opening	$(\perp, k, \perp)$, $1 \leq k < J_m - 1$; Equation (44).
5082	H	final opening	$(\perp, J_m - 1, \perp)$; Equation (44).
5101	S	fold challenge	$(\perp, k, i)$; one $\mathbb{K}$ challenge.
5102	S	OOD point	(a, k, c) ; one $\mathbb{K}$ coordinate, with c increasing.
5103	S	glue challenge	$(\perp, k, i)$ as fixed below; one $\mathbb{K}$ challenge.
5104	S	PoW seed	$(\perp, k, i)$; 16 bytes; request auxiliary payload C .
5105	S	query draw	$(\perp, k, a)$; one $\mathbb{K}$ challenge for attempt a .
5106	S	query alpha	$(\perp, k, c)$; one $\mathbb{K}$ challenge.

The GKR ranges are black-box namespaces, not permission to use implementation-defined bytes. The fixed GKR protocol must publish a ledger with the same columns as this table. The fixed 3001 event resolves the PCS-level phase- c placement ambiguity in Gap G1; the rest of that gap remains GKR-protocol work and no protocol may reassign the domain code. The v1 outer row LinCheck omits the redundant `presum.claimed_sums` field and derives $[e_d + 1]$, so domain code 3081 is reserved and MUST NOT occur. The ledger first consumes π_{GKR} to derive (ω_ξ, σ) and then, with no intervening absorb or squeeze, consumes π_{MLE} to derive (r_h, θ, η) . It must require $\theta = \widetilde{\omega}_\xi(r_h)\eta$ before returning the successor claim $\widetilde{H}_{\mathbb{K}}(r_h) = \eta$. The semantic P5/V5–P6/V6 split does not create a transcript event. An absent or incomplete fixed ledger is a hard reject.

6.6 Transcript transitions and replay

Model the transcript state as the byte string hashed by an unkeyed BLAKE3 hasher. It starts empty. The domain/version value 0001 and statement value 0002 are absorbed, in that order, before any proof-dependent squeeze. The PIOP-supplied linear-claim statement digest is

$$d_{\text{claim}} := \text{BLAKE3}_{256}(\text{hex}^{-1}(46325a2d5043532f6170702d73746174656d656e742f763100) \parallel \text{linear_claim_statement_bytes}). \quad (51)$$

For v1 byte compatibility, Equation (51) retains the legacy ASCII domain literal F2Z-PCS/app-statement/v1 followed by one NUL byte. Changing that literal requires a new wire version. The two initial payloads are exactly

$$\begin{aligned} \text{DomainV1} &:= \text{hex}^{-1}(46325a5043530000) \parallel \text{LE}_{16}(1), \\ \text{StatementV1} &:= d_{\text{claim}}[32] \parallel \text{LE}_{32}(d_r) \parallel \text{LE}_{32}(d_c) \parallel \text{LE}_{32}(m_f) \\ &\quad \parallel \text{Enc}_{\mathbb{K}}(g) \parallel \rho[32]. \end{aligned} \quad (52)$$

Thus DomainV1 and StatementV1 have 10 and 92 bytes, respectively. The host must make the linear-claim statement byte string in Equation (51) canonical. That string must bind $\text{Vec}_{\mathbb{F}_q}(v)$, $\text{Enc}_{\mathbb{F}_q}(\mu)$, the configured shapes and coordinate orders of F and H , and a virtualization-mode byte.

The vector encoding is $\text{LE}_{32}(n_h) \parallel \prod_{z=0}^{n_h-1} \text{Enc}_{\mathbb{F}_q}(v[z])$. The direct mode binds no matrix and requires the effective map to be the identity; the virtualized mode also binds the canonical dimensions and entries of M . Consequently an omitted map and an explicitly configured nonidentity map cannot share a statement digest.

After the final input-Sumcheck challenge, the verifier derives and absorbs

$$\text{LinCheckSeamV1} := \text{LE}_{32}(d_r) \parallel \text{LE}_{32}(d_c) \parallel \text{Vec}_{\mathbb{F}_q}(s_h) \parallel \text{Enc}_{\mathbb{F}_q}(\kappa) \parallel \text{Enc}_{\mathbb{F}_q}(\tau). \quad (53)$$

Here $\text{Vec}_{\mathbb{F}_q}(s_h)$ has its canonical u_{32} count m_h followed by the m_h canonical coordinates in transcript order. Equation (53) has exactly $44 + 16m_h$ bytes. It is absorbed under derived domain code 1003 after the last accepted domain code 1101 challenge and before the domain code 2001 record. The verifier derives $\kappa = \widetilde{v}(s_h)$, $a_r = \text{eq}_{\mathbb{F}_q}(r, s_r)$, and $a'_c = \kappa \text{eq}_{\mathbb{F}_q}(c, s_c)$; none of these dense factor vectors is serialized in the seam.

Absorption appends exactly Equation (50). A squeeze under base domain code $\tau < 0x8000$, verifier-derived scope (j, k, i) , output length n , and auxiliary request payload a first sets $q := \text{LE}_{32}(n) \parallel a$ and then performs the transition

$$\begin{aligned} q &:= \text{LE}_{32}(n) \parallel a, \\ \text{tr}' &:= \text{tr} \parallel D_{\tau, j, k, i} \parallel \text{LE}_{64}(\text{bytelen}(q)) \parallel q, \\ z &:= \text{BLAKE3-XOF}(\text{tr}')[0 \dots n], \\ \text{tr}'' &:= \text{tr}' \parallel D_{\tau \text{OR} 0x8000, j, k, i} \parallel \text{LE}_{64}(n) \parallel z. \end{aligned} \quad (54)$$

Equation (54) uses a clone/finalize-XOF of the incremental hasher after the request bytes; it does not reset or key the hasher. A $\mathbb{K}$ challenge has $n = 16$ and decodes by Section 6.2. A vector challenge is a sequence of scalar squeezes, not one long squeeze. Unless its registry row says otherwise, the scalars use consecutive i values and the auxiliary request payload a is empty.

Domain code 1101 samples one unbiased $\mathbb{F}_q$ challenge. For attempts $a = 0, 1, \dots$, apply Equation (54) with $n = 16$ and auxiliary payload $\text{LE}_{32}(a)$. Interpret the raw result as a little-endian u_{128} , clear its high 28 bits, and accept the resulting candidate iff it is less than $q = 2^{100} - 15$; otherwise continue with the next attempt. Every request and raw domain code 9101 result remains in the transcript, including rejected attempts. Reject before attempt 2^{32} ; the counter must not wrap.

Domain code 5103 uses its one-byte glue kind, and domain code 5104 uses the PoW context C ; domain codes 1101, 5103, and 5104 are the only core squeezes with nonempty auxiliary payloads.

Immediately after domain codes 0001 and 0002, the verifier runs exactly m_h input-Sumcheck rounds. Starting with $\sigma_{-1}^{\text{shape}} = \mu$, round i reads one payload, requires $g_i^{\text{shape}}(0) + g_i^{\text{shape}}(1) = \sigma_{i-1}^{\text{shape}}$, absorbs the payload under domain code 1002, and then samples $s_{h,i}$ under domain code 1101 at the same scope, and sets $\sigma_i^{\text{shape}} = g_i^{\text{shape}}(s_{h,i})$. No event may intervene between that read and its accepted challenge. After the final round it sets $\tau = \sigma_{m_h-1}^{\text{shape}}$ and $\kappa = \tilde{v}(s_h)$, then immediately absorbs the derived domain code 1003 seam. The remaining core must discharge $\kappa \tilde{H}_q(s_h) = \tau$; the seam does not assert that the original v equals the derived equality weights.

On a normal NARG read, the verifier derives the next expected domain and payload decoder from the ledger, validates the canonical payload, and immediately applies Equation (50). A PoW nonce is the sole deferred NARG read: the verifier parses its expected canonical payload, permits no intervening event, checks Equation (56), and absorbs it only on success. On a Hint read, it derives the next expected decoder and validates the canonical payload but never absorbs it. It must complete the hint predicate before the next squeeze. In particular, it reads and absorbs 2001, then reads 2081 and requires $|\mu| = 2^{d_c}$, $0 \leq \mu_c < 2^{127}$ and $g^{\mu_c} = \nu_c$ for every c , and $\sum_c a'_c \pi_q(\mu_c) = \tau$ before the first 2101 event. NARG replay follows verifier-event order, never Rust structure-field order. Acceptance requires no deferred record or pending hint and exact logical and byte exhaustion of both streams and every nested slice.

After the adjoint, both parties derive the same canonical $\text{RingDecomp}(a_f)$ and require $A_f[v, j] = \sum_{\theta \in \Theta} A_\theta[v] B_\theta[j]$. For each $\theta \in \Theta$ in order, the verifier reads and absorbs one domain-code 4001 vector and requires $|s_\theta| = 128$. After all such records, and before domain code 4101, it checks the single anchor equation

$$\sum_{\theta \in \Theta} \sum_{v=0}^{127} A_\theta[v] s_{\theta,v} = \eta.$$

It then squeezes the seven coordinates of r'' under domain code 4101, derives $(\omega_\theta, \beta_\theta)$ for every term, and squeezes one domain-code 4102 coefficient λ_θ per term in the same order. Finally it derives $B = \sum_\theta \lambda_\theta \omega_\theta$ and $\beta = \sum_\theta \lambda_\theta \beta_\theta$. No post-adjoint LinCheck or coefficient-Sumcheck event exists; domain codes 4002 and 4103 are reserved.

6.7 Grinding and query derivation

Let kind byte zero denote a fold grind and kind byte one a query grind. For kind K_{pow} , level k , round i , and derived difficulty d , define the exact PoW context

$$C := \text{ASCII}(\text{F2Z-PCS/PoW/v1}) \parallel \text{LE}_8(K_{\text{pow}}) \parallel \text{LE}_{32}(k) \parallel \text{LE}_{32}(i) \parallel \text{LE}_{16}(d). \quad (55)$$

In Equation (55), $i = \perp_{32}$ for a query grind. The seed is the 16-byte output of Equation (54) under domain code 5104, the same scope, and auxiliary request payload C . After obtaining the seed, the verifier reads the expected 5007 or 5008 NARG nonce payload without absorbing it and requires, for the scheduled positive difficulty d ,

$$\text{LZ}(\text{BLAKE3}_{256}(C \parallel \text{seed}_{16} \parallel \text{LE}_{64}(n))) \geq d, \quad d > 0. \quad (56)$$

LZ in Equation (56) counts zero bits from the most-significant bit of digest byte zero, then proceeds toward the least-significant bit and subsequent bytes. Any satisfying nonce is accepted; minimality is not checked. Only after the predicate succeeds does the verifier absorb the nonce under its verifier-derived domain, before any dependent squeeze. Difficulty zero has no predicate invocation, seed squeeze, or nonce record in this wire format. The exact scheduled difficulties are

$$d_k^{\text{query}} := g_k = 16, \quad d_{k,i}^{\text{fold}} := \max(f_k - i, 0). \quad (57)$$

Equation (57) is evaluated with ordinary nonnegative integers. After the preceding sumcheck message and immediately before fold challenge (k, i) , perform both the domain code 5104 seed squeeze and domain code 5007 nonce record if $d_{k,i}^{\text{fold}} > 0$; if the difficulty is zero, perform neither event. For $0 \leq k < J_m - 1$, query event k occurs after the domain code 5003 commitment root for level $k+1$, all o_{k+1} OOD bindings for that root, and their glue challenges, and immediately before the domain code 5105 draws that open level k . Query event $J_m - 1$ instead occurs after the domain code 5006 final y_r , with no new root or OOD binding, and immediately before the final-level draws. There is exactly one query nonce at every level. Consequently the exact number of nonce records is

$$N_{\text{pow}} = J_m + \sum_{k=0}^{J_m-1} \min(k_k, f_k). \quad (58)$$

Missing, extra, reordered, or trailing nonces violate Equation (58) and reject.

For level k , each 5105 event yields $r \in \mathbb{K}$ and proposes

$$q := r_{10} \bmod 2^{h_k + r_k}. \quad (59)$$

The verifier repeats Equation (59) until Q_k distinct positions exist, discarding duplicates, and then sorts the positions in increasing order. It then derives the domain code 5106 alphas before reading and checking the opening.

6.8 Recursive-opening replay ledger

This subsection fixes the complete replay order for the recursive opening. The four canonical message kinds and scopes for domain code 5004 are

0x00	Start	$(\perp, 0, \perp)$	exactly once,	
0x01	FoldNext	$(\perp, k, j)$	$0 \leq j < k_k$,	
0x02	OodIntro	$(a, k, \perp)$	$1 \leq k < J_m, 0 \leq a < o_k$,	
0x03	QueryIntro	$(\perp, k, \perp)$	$0 \leq k < J_m - 1$.	(60)

Equation (60) is the complete kind registry for domain code 5004. Each payload is the one-byte kind followed by the two canonical $\mathbb{K}$ elements in the registry. Any other kind/scope pair rejects.

The outer VerifyF2Z caller uses the deterministic coefficient function B derived locally in P8/V8, with domain size 2^{m_p} ; B is never densely serialized. The caller alone absorbs domain code 5001 with payload $\text{LE}_{32}(m_p) \parallel \text{Enc}_{\mathbb{K}}(\beta)$ and then invokes RecVerify. That helper starts after domain code 5001: its first event reads domain code 5002, requires the root to equal ρ , absorbs it, and then reads and absorbs the Start domain code 5004 message.

For each $k = 0, \dots, J_m - 1$ in increasing order, it then performs the following ledger without interleaving another event.

1. For $j = 0, \dots, k_k - 1$, perform the optional fold grind exactly as in Section 6.7, squeeze domain code 5101 at $(\perp, k, j)$, then read, validate, and absorb the FoldNext domain code 5004 message at the same scope.
2. If $k < J_m - 1$, read and absorb the domain code 5003 root at $(\perp, k+1, \perp)$. For each $a = 0, \dots, o_{k+1} - 1$, squeeze the n_{k+1} domain code 5102 coordinates at scopes $(a, k+1, c)$ for $c = 0, \dots, n_{k+1} - 1$; read and absorb domain code 5005 at $(a, k+1, \perp)$; read and absorb the OodIntro domain code 5004 message at that scope; and squeeze domain code 5103 at $(\perp, k+1, a)$ with the one-byte auxiliary kind 00.
If instead $k = J_m - 1$, read and absorb domain code 5006 at $(\perp, k, \perp)$, requiring exactly 2^{h_k} canonical $\mathbb{K}$ elements.
3. Perform the query grind at $(\perp, k, \perp)$ with difficulty 16 and absorb its validated domain code 5008 nonce. Starting with attempt $a = 0$, squeeze domain code 5105 at $(\perp, k, a)$ and increment a after every squeeze, including a duplicate, until Q_k distinct positions have been obtained. Reject

rather than wrap if the next attempt would exceed $2^{32} - 1$. Sort the positions, then squeeze exactly $\lceil \log_2 Q_k \rceil$ domain code 5106 alphas at $(\perp, k, c)$ for consecutive c .

Read exactly one opening Hint using domain code 5080 for $k = 0$, 5081 for $0 < k < J_m - 1$, or 5082 for $k = J_m - 1$. Check Equations (44)–(45) and the root before any later event. For $k < J_m - 1$, next read and absorb the QueryIntro domain code 5004 message. At every level, including the final level where no such message exists, squeeze the query glue domain code 5103 at $(\perp, k, o_k)$ with one-byte auxiliary kind 01. At the final level this challenge is used directly in the residual check.

The fixed record/event cardinalities are therefore

$$\begin{aligned}
 \#5002 &= 1, & \#5003 &= J_m - 1, & \#5005 &= \sum_{k=1}^{J_m-1} o_k, \\
 \#5004 &= 1 + \sum_{k=0}^{J_m-1} k_k + \sum_{k=1}^{J_m-1} o_k + (J_m - 1), & \#5006 &= 1, & \#508* &= J_m, \\
 \#5101 &= \sum_{k=0}^{J_m-1} k_k, & \#5102 &= \sum_{k=1}^{J_m-1} o_k n_k, & \#5103 &= J_m + \sum_{k=1}^{J_m-1} o_k, \\
 \#5106 &= \sum_{k=0}^{J_m-1} \lceil \log_2 Q_k \rceil.
 \end{aligned} \tag{61}$$

The cardinalities in Equation (61) are exact. Here #508* is one opening Hint per level. The number of domain- 5105 events is the transcript-derived number of attempts, all of which appear in the audit log. Nonce counts remain Equation (58). A different order, scope, kind, cardinality, or unread field rejects.

6.9 Parser, implementation, and conformance requirements

A v1 implementation MUST:

1. validate magic, version, flags, all public-parameter-, statement-, and transcript-derived bounds, and all reserved fields before allocating or doing proof work;
2. use checked arithmetic and checked conversion for every length and index, reject noncanonical field values, and return a decoding error rather than panic on adversarial bytes;
3. encode the NARG stream in the exact absorb order and the Hint stream in its exact check order, with independent cursors;
4. replace opaque recursive-proof serialization with the field records and exact order in Sections 6.5 and 6.8; and
5. reject unless every outer, subordinate, recursive, row, and Merkle-path slice is exactly exhausted.

Conformance fixtures MUST include canonical primitive, domain, and payload hex, one complete $N/H/\text{HostEncode}_{v1}$ proof, the ordered transcript audit log, every squeezed challenge, PoW seeds/nonces and query positions, and all four lengths from Equation (47). Negative fixtures MUST cover an unknown version, wrong endian or channel, record reordering, truncation and suffix bytes, noncanonical scalars, count overflow, a wrong input-Sumcheck round polynomial, wrong initial target, reordered 1002/1101 events, a changed domain code 1003 seam, a failed or late μ/ν hint, altered PoW scope/difficulty/nonce, and missing or extra recursive fields. Fixtures must also cover at least one rejected domain code 1101 sampling attempt and its transcript continuation. Ring-switch fixtures must additionally cover a malformed or reordered 4001 term, a failed anchor equation, and reordered or missing 4101/4102 challenges. Native, recursive, and independent reference verifiers must produce byte-identical transcript audit logs.

The repository file `wire-v1-vectors.txt` is the normative initial known-answer set for the primitive container/domain/payload codec, Merkle row/tree/multiproof codec, and PoW predicate at difficulties 1, 7, 8, and 16. A conforming implementation must reproduce it exactly and add a complete proof/transcript fixture before claiming end-to-end v1 interoperability.